

How Datadog Built a Custom Database to Ingest Billions of Metrics Per Second



BYTEBYTEGO
NOV 04, 2025



234



4



10

Share



ByteByteGo Newsletter
By Alex Xu



Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe

[Build a more sustainable on-call experience \(Sponsored\).](#)

ByteByteGo

@bytebytego399569



A Practical Guide to Sustainable On-Call Rotations



BEST PRACTICES EBOOK



ByteByteGo Newsletter
By Alex Xu



Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe

Keeping your systems reliable shouldn't come at the expense of your team. This practical guide from Datadog shows how to design sustainable on-call processes that reduce burnout and improve response.

Get step-by-step best practices

ByteByteGo

@bytebytego399569



- Cut alert noise with sign
- Streamline response wit
- Design rotations that su



ByteByteGo Newsletter
By Alex Xu



Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe

bility

Disclaimer: The details in this post have been derived from the details shared online by the Datadog Engineering Team and the P99 Conference Organizers. All credit for the technical details goes to the Datadog Engineering Team. The links to the original articles and sources are present in the references section at the end of the post. We've attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to fix them.

In the world of cloud monitoring, Datadog operates at a massive scale. The company's monitoring platforms must ingest billions of data points every single second from millions of hosts around the globe.

This constant flow of information creates an immense engineering challenge: how do you store, manage, and query this data in a way that is not only lightning-fast but also cost-effective?

For the Datadog Engineering Team, the answer was to build their own solution from the ground up.

In this article, we will understand how the Datadog engineering team built Monocloud, their custom-built time series storage engine to power their real-time metrics platform. This article analyzes the technical decisions and clever optimizations behind the database.

ByteByteGo

@bytebytego399569



[Move faster with](#)
[\(Sponsored\)](#)

[trust](#)



ByteByteGo Newsletter
By Alex Xu



Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe

Sonar

CODE YOU CAN TRUST

Verify your AI code
with SonarQube

Learn More



AI is speeding things up, but all that speed creates a bottleneck — who's verifying the quality and security? Do

ByteByteGo
@bytebytego399569



risks slip past.

Sonar's automated review gives you confidence in the quality of code, human or AI-written.

With SonarQube, your team



ByteByteGo Newsletter

By Alex Xu

- Use AI without fear: Get the latest on AI-powered code security.
- Fix issues early: Detect and apply automated fixes before you merge.
- Maintain your standards: Ensure all code meets your team's quality bar, every time, for long-term code health.

[Get started with SonarQube today](#) to fuel AI-enabled development and build trust in all code.

High-Level Metrics Platform Architecture

Before diving into the custom database engine, it is important to understand where it fits.

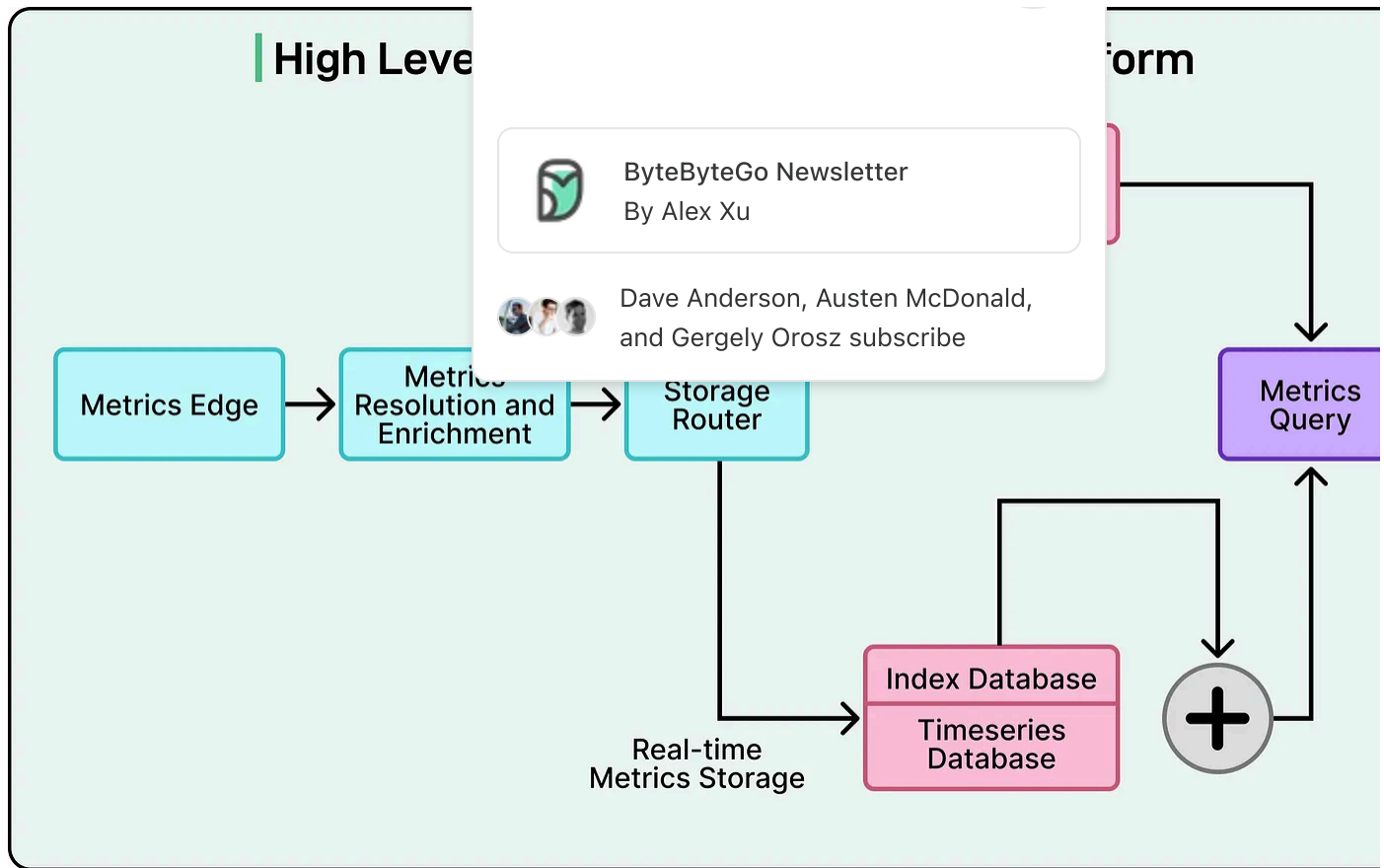
Their custom engine, named Monocle, is just one specialized component within a much larger “Metrics Platform.” This platform is the entire system responsible for collecting, processing, storing, and serving all of its customer metrics.

The journey of a single data point begins at the “Metrics Edge.” This component acts as the front door, receiving the flood of data from millions of customer systems. From there, it is passed to a “Storage Router.” Just as the name suggests, this router's main job is to process the incoming data and intelligently decide where it needs to be stored.

ByteByteGo

@bytebytego399569





This is where Datadog's first major design decision becomes clear.

The Datadog Engineering Team recognized that not all data queries are the same. An engineer asking for a performance report from last year has very different needs than an automated alert checking for a failure in the last 30 seconds. To serve both, they split their storage into two massive, specialized systems.

- The first system is the Long-Term Metrics Storage. This system is like a vast historical archive. It is built for heavy-duty analytical queries, where an engineer might want to analyze a performance trend over many months or even years.
- The second system is the Real-Time Metrics Storage, which is the primary focus of this article. This system is a high-speed, high-performance engine that holds only the most recent data, roughly the latest 24 hours. While that might not sound like a lot, this system handles the vast majority of the platform's workload: 99% all queries. These are the dashboards and

ByteByteGo

@bytebytego399569



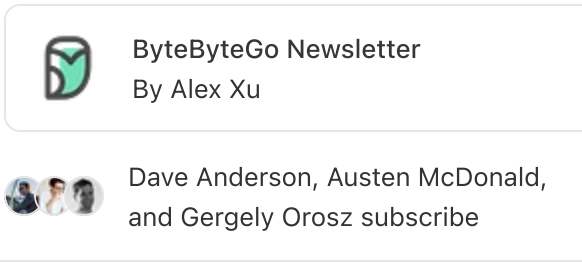
dashboards and

critical, automated monitoring
screen update in real-time

a graph on their

A time series data point has

- **The Data:** This is the actual value, for example [12:00:01 PM, 80].
- **The Metadata:** These are the labels, or “tags,” that describe what the data is, such as service:api, host:server-10, or region:us-east-1.



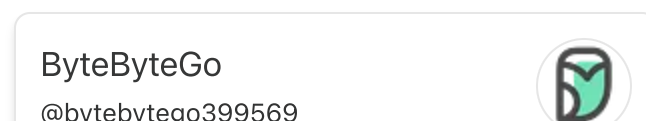
The Datadog Engineering Team made the critical decision to store these two parts separate, specialized databases:

- The Index Database stores only the tags (the metadata). Its only job is to be incredibly fast at finding data. When a user asks, “Show me the CPU for all services in us-east,” this index database instantly finds all the unique data streams that match those tags.
- The Real-Time Timeseries Database (RTDB) stores only the timestamps and values. It is a highly optimized storage that, after getting the list of streams from the Index Database, just retrieves the raw numbers.

Using Kafka

Perhaps the most unique architectural decision the Datadog Engineering Team made is how their database clusters are organized. In many traditional distributed databases the server nodes (the individual computers in the cluster) constantly talk to each other. They “chatter” to coordinate who is doing what, to copy data between themselves (a process called replication), and to figure out what to do if one of them fails.

Datadog’s RTDB nodes do not do this.



Instead, the entire system is a central place where all new data is written. This Kafka-centric design is what makes it so easy to see the diagram below that

Kafka acts as a central place where all new data is written. This Kafka-centric design is what makes it so easy to see the diagram below that

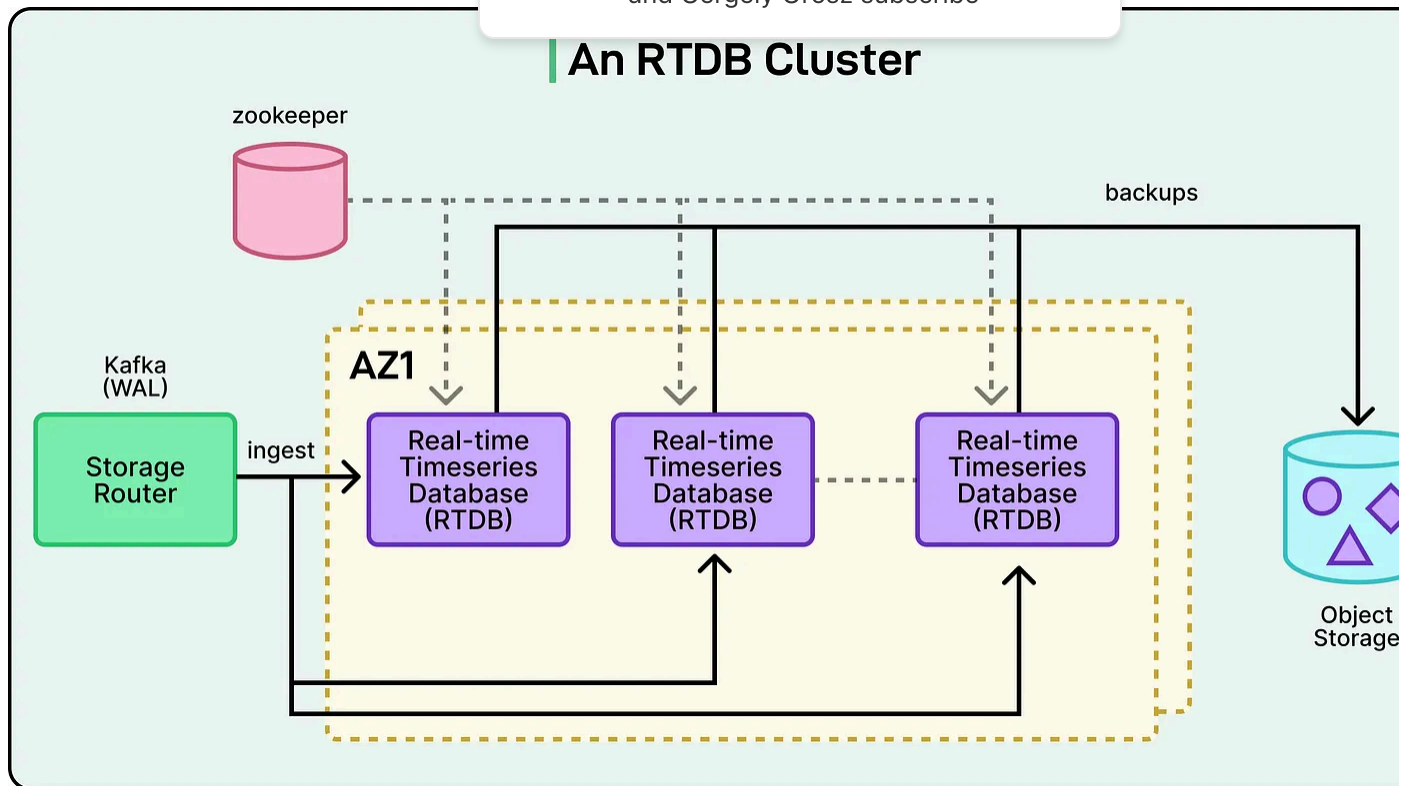
Kafka:



ByteByteGo Newsletter
By Alex Xu



Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe



The Datadog Engineering Team uses Kafka to perform three critical functions that database nodes would otherwise have to do themselves.

- First is Data Distribution. The data in Kafka is organized into “topics” (like metrics), which are split into smaller logs called “partitions.” The team designed the system in such a way that each RTDB database node is assigned its own specific set of partitions to read from. This neatly divides the billions of incoming data points without the nodes ever needing to coordinate.
- Second, Kafka acts as the Write-Ahead Log (WAL). This is a standard database concept for ensuring data consistency. In a distributed database, if a node fails, the “single source of truth.”

ByteByteGo
@bytebytego399569



Datadog creates a single source of truth. In the event of a hardware failure, no

data is lost. When the next known reading position

- Third, Kafka handles Replication of your data in different partitions, which allows for replication, which allows for multiple data centers (known as Availability Zones) out of the box.



ByteByteGo Newsletter
By Alex Xu



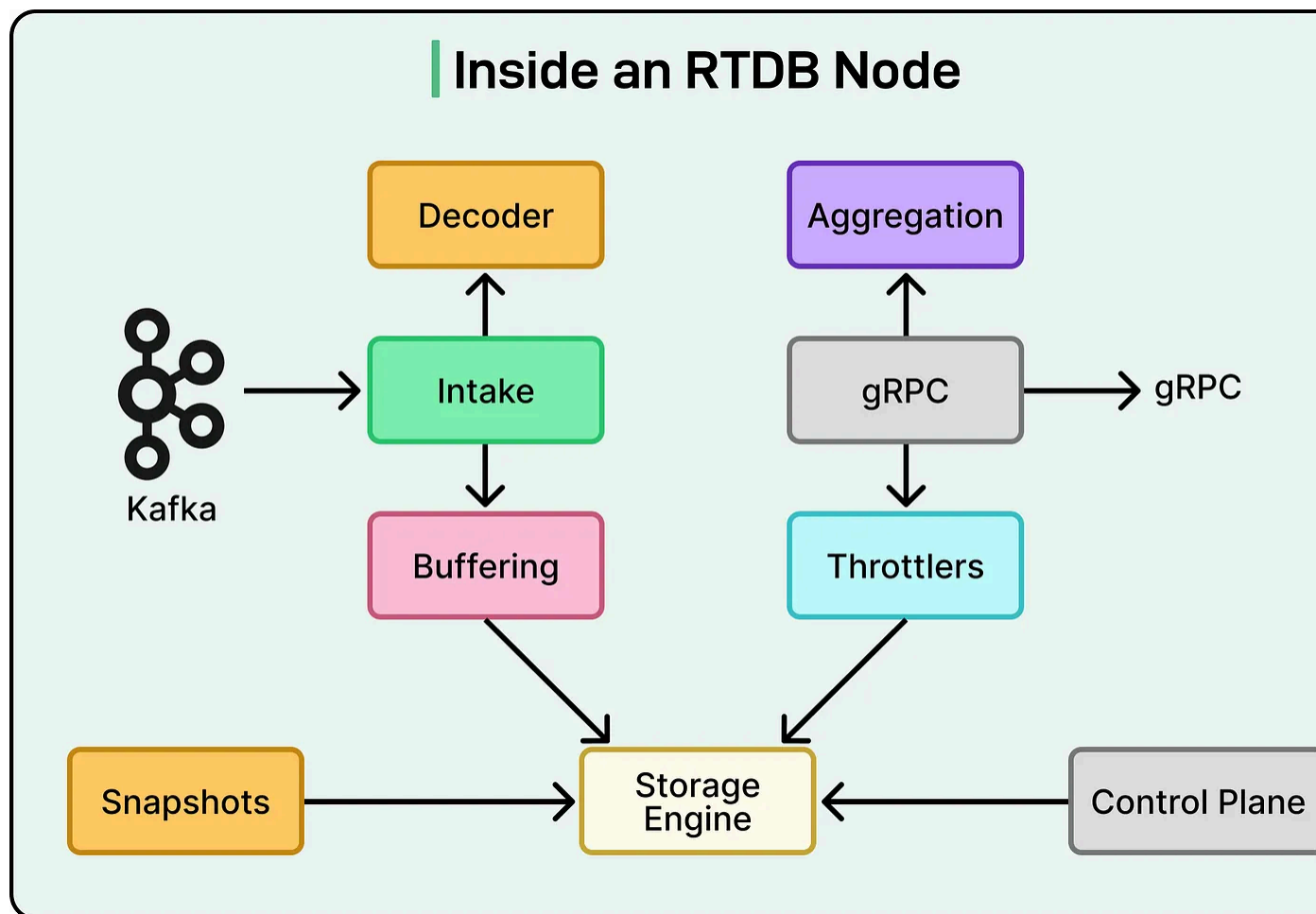
Dave Anderson, Austen McDonald,
and Gergely Orosz subscribe

Kafka, finds its last read position.

Each broker must have copies of all data. Kafka's built-in replication across different data centers allows for faster recovery in the event of a failure.

Monocle: The Custom Built Engine in Rust

At the heart of each RTDB node is Monocle, Datadog's custom-built storage engine. See the diagram below:



This is where the team's pursuit of performance gets truly impressive. While earlier versions of the platform used RocksDB, a popular and powerful open-source database engine, the team ultimately decided to build its own. By creating Monocle from scratch, they could tailor every single decision to their specific needs, unlocking a new level of efficiency.

Monocle is written in Rust, a modern programming language famous for its safety guarantees and "C-like" performance. It is built on Tokio, a popular framework in the Rust ecosystem for writing high-speed, asynchronous applications that can handle many tasks at once without getting bogged down.

The Core Data Model: Hashing Tags

Monocle's key innovation is its simple data model. As mentioned, a time series is defined by its tags, like "system.cpu.user", "host:web-01", and "env:prod". This set of tags is what makes a series unique. However, these tag sets can be long and complex to search.

The Datadog Engineering Team simplified this dramatically. Instead of working with these complex strings, Monocle hashes the entire set of tags for a series, turning it into a single, unique number. The database then just stores data in a simple map:

```
(Organization, Metric Name, Tag Hash) -> (A list of [Timestamp, Value] pairs)
```

This design is incredibly fast because finding all the data for any given time series becomes a direct and efficient lookup using that single hash. The separate Index Database is responsible for the human-friendly part: it tells the system that a query for "env: prod" corresponds to a specific list of "Tag Hashes."

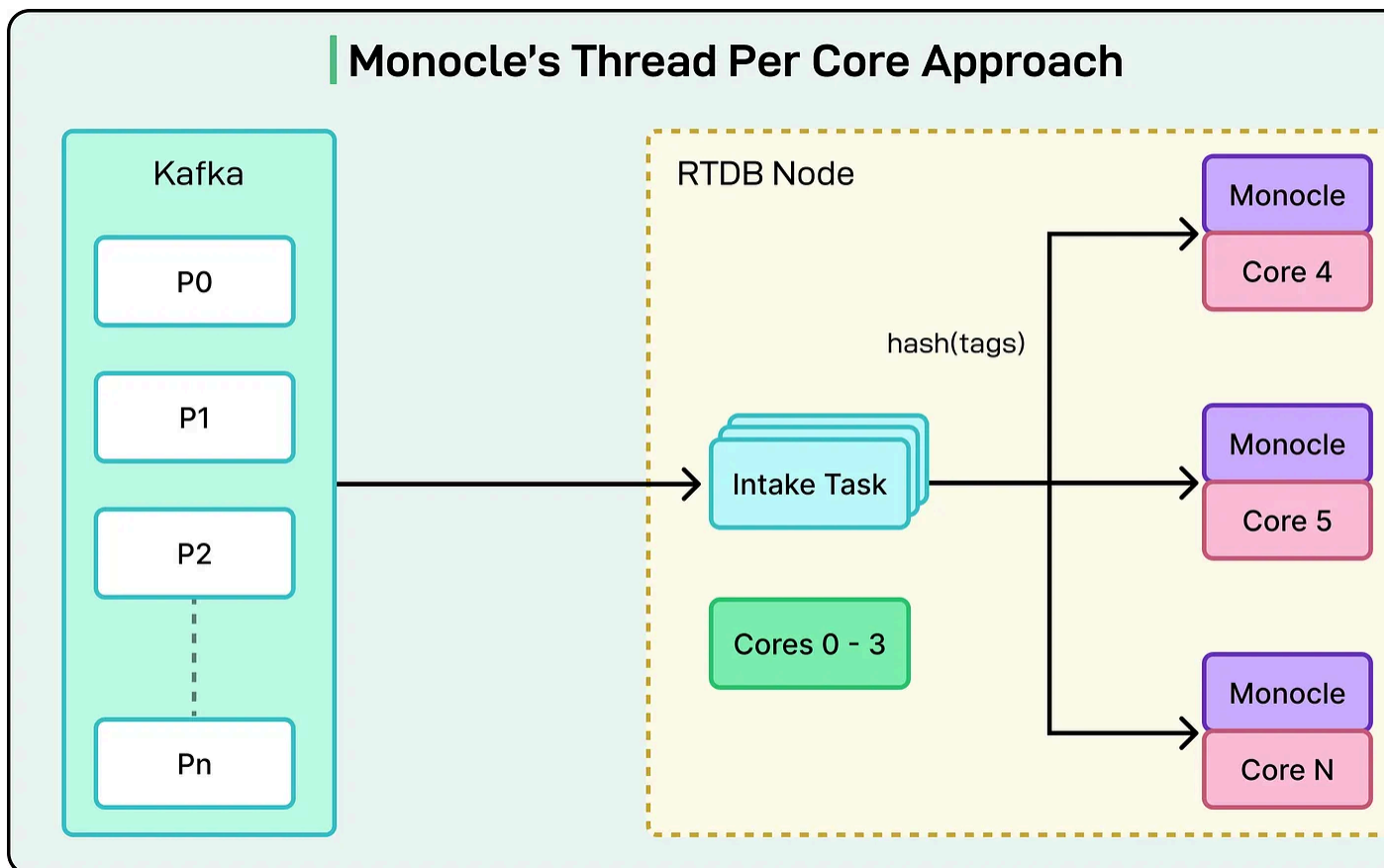
Inside Monocle

Monocle's speed comes from two main areas: its concurrency model and its storage structure.

Monocle uses what is known as a “thread-per-core” or “shared-nothing” architecture. You can imagine each CPU core in the server has its own dedicated worker, which operates in total isolation. Each worker has its own data, its own cache, and its own memory. They do not share anything.

When new data comes in from Kafka, it is hashed. The system then sends that data to the specific queue for the one worker that “owns” that hash. Since each worker is the only one who can ever access their own data, there is no need for locks, no coordination, and no waiting. This eliminates a massive performance bottleneck common in traditional databases, where different threads often have to wait for each other to access the same piece of data.

See the diagram below:



Monocle's storage layout is a Log-Structured Merge-Tree (LSM-Tree). This is a design that is extremely efficient for write-heavy workloads like Datadog's.

Here are the main concepts associated with LSM Trees:

- **Memtable:** All new data is batched in memory in a structure called a Memtable
- **Flushing:** When the Memtable is full, it is “flushed” to disk as a complete, read only file. The system never goes back to modify old files.
- **Compaction:** Over time, a background process called compaction tidies up by merging these small files into larger, more organized ones.

The Datadog Engineering Team added two critical optimizations to this design:

- **Arena Allocator:** Normally, when a Memtable is flushed, the database would have to free millions of tiny objects from memory, which is a slow process. Monocle uses a custom arena allocator. This means the Memtable gets one giant chunk of memory (an arena). When it is done, the entire chunk is freed all at once, which is vastly more efficient.
- **Time-Based File Pruning:** Since 99% of queries are for recent data, Monocle takes advantage of this. Each file on disk has a known time range (for example, 10:00 AM - 10:10 AM). When a query comes in for the last hour, Monocle can instantly prune (ignore) all the files that are not from that time window. This reduces the number of files it has to read to find the answer.

Staying Fast Under Pressure

Handling so many queries with the “thread-per-core” design creates a unique challenge.

Since a query is fanned out to all workers, it is only as fast as its slowest worker. If a worker is busy performing a background task, like a heavy data compaction, it can't

the entire query for all the other workers. This is a classic computer science problem known as head-of-line blocking.

To solve this, the team built a two-layer system to manage the query load and stay responsive.

- The first layer is Admission Control. This acts as a simple “gate” at the front door. If the system detects it is under extreme load (for example, it is falling behind on reading data from Kafka or is low on memory), this gate will simply stop accepting new queries. This protects the database from being overwhelmed.
- The second is a smarter Cost-Based Scheduling system. This layer uses a well-known algorithm called CoDel (Controlled Delay) to actively manage latency. It can prioritize queries based on their cost (how much work they need to do) and ensure that even under heavy, unpredictable query bursts, the database remains responsive and does not grind to a halt.

Conclusion

The Datadog Engineering Team’s work on Monocle is far from over. They are already planning the next evolution of their platform, which involves two major changes.

- **Smarter Routing:** The current system, while effective, is relatively static. The team is developing a dynamic, load-balancing system that will allow the cluster to move data around in real-time. This would let the database automatically adapt to query “hot spots,” for instance, when many engineers are suddenly querying the same small set of metrics during an incident.
- **Colocate Points and Tags:** This is a fundamental shift. The team plans to merge their two separate databases (the Index Database and the RTDB) into a single, unified system. Instead of just storing tag hashes, the new database will store the full tag strings right alongside their corresponding timestamps and values.

To achieve this, the team will move to a columnar database format.

In a columnar database, data is stored by columns instead of rows. This means a query can read only the specific tags and values it needs, which is a massive speedup for analytics.

This is a complex undertaking that will likely require a complete redesign of their “thread-per-core” model, but it highlights Datadog’s drive to push the boundaries of performance.

References:

- [Timeseries Storage at Ludicrous Speed](#)
- [Timeseries Indexing at Scale](#)
- [Evolving our real-time time series storage again: Built in Rust for performance scale](#)

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing sponsorship@bytebytego.com.





234 Likes • 10 Restacks

Discussion about this post

Comments Restacks



Write a comment...



cch  Nov 8, 2025

Super interested in hearing more from their team in the future about the part "rethinking the separation between indexing and timeseries storage in our metrics platform" they mentioned their blog.

 LIKE (1)  REPLY



Jankins Xingole Jankins Xingole Apr 19

The Arena Allocator optimization technology is also applied to the LevelDB, if my memory is r

 LIKE  REPLY



2 more comments...